

Pruebas Unitarias Automatizadas

Proyecto Teleprogreso S.A.

Control de personal y supervisión de rutas — pytest + Vitest



```
$ pytest tests/ -v  
===== 23 passed in 2.15s =====
```

```
$ npm test  
Test Files  4 passed (4)  
Tests      22 passed (22)
```

45 pruebas unitarias OK

Nuestro stack tecnológico



Backend — FastAPI

- Python 3 + FastAPI y SQLAlchemy (async)
- PostgreSQL + PostGIS y Alembic
- Autenticación JWT con 4 roles: técnico, supervisor, admin y gerente



Frontend — React 18

- Vite como build tool (PWA)
- React Router + Axios contra la API
- Componentes propios: badges, modales, cronómetro de pausas y mapas de ruta

MÓDULOS DEL SISTEMA

tareas

inventario

asistencia

rutas

empleados

métricas

Frameworks de prueba en Python (backend)

pytest

El estándar de facto: sintaxis simple con assert, fixtures potentes y más de 12,000 plugins.

ELEGIDO

unittest

Incluido en la librería estándar; estilo JUnit basado en clases, notablemente más verboso.

nose2

Sucesor de nose; funcional pero con comunidad pequeña y desarrollo lento.

doctest

Pruebas dentro de docstrings; útil solo para ejemplos y casos muy simples.

Robot Framework

Orientado a pruebas de aceptación por palabras clave; curva de aprendizaje alta para unit testing.

Frameworks de prueba en JavaScript (frontend)

Vitest

ELEGIDO

Runner nativo de Vite: reutiliza su configuración, muy rápido y con API compatible con Jest.

React Testing Library

ELEGIDO

Prueba componentes como los ve el usuario (roles y textos). Complementa al runner.

Jest

El más popular del ecosistema, pero exige configuración extra (babel/transform) en proyectos Vite.

Mocha + Chai

Muy flexible, pero hay que armar runner, asserts y mocks por separado.

Cypress / Playwright

Excelentes para pruebas E2E en navegador real; no están enfocados en pruebas unitarias.

La combinación elegida: Vitest como runner + Testing Library para renderizar y consultar componentes.

¿Por qué elegimos pytest?

pytest

+52M

descargas mensuales en PyPI

+12,000

plugins disponibles



Cero fricción

Ya estaba en nuestro requirements.txt: no agregó dependencias nuevas al proyecto.



Fixtures reutilizables

Tokens JWT por rol, empleados mock y TestClient definidos una sola vez en conftest.py.



Async de primera clase

pytest-asyncio encaja con el stack async de FastAPI y SQLAlchemy.



Respaldado por FastAPI

La documentación oficial de FastAPI enseña a probar con pytest y TestClient.

¿Por qué elegimos Vitest + Testing Library?



Reutiliza la configuración de Vite

El mismo vite.config.js sirve para dev y tests: mismo transform de JSX, cero duplicación.



Velocidad

Hasta 4x más rápido que Jest en proyectos Vite; nuestros 22 tests corren en ~4 segundos.



API compatible con Jest

describe / it / expect: la curva de aprendizaje para el equipo fue mínima.



Prueba como el usuario

Testing Library consulta por roles y textos visibles, no por detalles internos del DOM.

vitest

4x

más rápido que Jest sobre Vite

jsdom

simula el navegador: sin Chrome, sin servidor

Ejemplos backend · seguridad y JWT

```
# tests/test_security.py
def test_verify_password_incorrecta():
    hashed = hash_password("Clave123")
    assert verify_password("Otra456",
                           hashed) is False

def test_token_revocado_no_es_valido():
    token = create_access_token(7, rol)
    revoke_token(token) # logout
    with pytest.raises(JWTError):
        decode_access_token(token)

tests/test_security.py ..... [100%]
8 passed in 1.42s
```



Hashing bcrypt

La contraseña nunca se guarda en texto plano; el salt aleatorio hace únicos los hashes.



Payload del JWT

Cada token incluye sub (id), rol, exp e iat; un token malformado lanza JWTError.



Logout real

Un token revocado queda en la lista negra y es rechazado aunque no haya expirado.

8 pruebas de hashing y tokens + 8 de validación Pydantic (test_schemas_tarea.py)

Ejemplos backend · endpoints protegidos

401

Sin token / token inválido

Llamar a /tarefas o /asistencia sin Authorization es rechazado.

403

Rol sin permiso

Un técnico no puede crear tareas; una cuenta inactiva no entra.

200

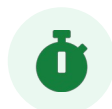
Token válido

Con JWT correcto, /auth/me devuelve el perfil y el rol.

Sin base de datos real: la dependencia `get_db` se reemplaza con `app.dependency_overrides` y un `AsyncMock` que devuelve empleados simulados. Es la técnica oficial de FastAPI: los tests corren en cualquier máquina sin levantar PostgreSQL ni Docker.

7 pruebas en `test_auth_protection.py` + 8 en `test_schemas_tarea.py` (enums, defaults y campos obligatorios)

Ejemplos frontend · componentes y hooks



`useTimer.test.jsx`

7 ✓

Cronómetro de pausas con fake timers: avanza el reloj 5 s virtuales, verifica que se detenga en pausa y el formato HH:MM:SS.



`StockBadge.test.jsx`

5 ✓

Lógica de inventario: sin stock, stock bajo y el caso borde disponible === mínimo que nadie había considerado.



`Badge.test.jsx`

5 ✓

Mapeo automático estado → color (completado = verde, urgente = rojo) y variantes explícitas.



`Modal.test.jsx`

5 ✓

Apertura y cierre: botón ×, tecla Escape, click fuera cierra pero click dentro no.

Resultados de la ejecución

23

pruebas backend

pytest · 2.15 s

22

pruebas frontend

Vitest · 4.43 s

45

pruebas en total

0 fallos

<7s

ejecución completa

ideal para CI/CD



```
$ cd backend && pytest tests/ -v          # 23 passed in 2.15s
$ cd frontend && npm test                  # 22 passed (4 files)
Un solo comando por lado. Sin PostgreSQL, sin navegador, sin Docker.
```

Lo que observamos usándolas

VENTAJAS



Retroalimentación inmediata

En modo watch, romper una función enciende el test en rojo al guardar el archivo.



Detección temprana de casos borde

Escribir tests nos hizo definir qué pasa cuando disponible === mínimo.



Documentación viva

Los nombres de los tests describen el comportamiento esperado del sistema.



Independencia total

Los mocks eliminan la base de datos y el navegador: corren en cualquier máquina.

DESVENTAJAS



Configuración inicial

Integrar Vitest + jsdom + Testing Library tomó tiempo de lectura de docs.



Curva con mocks async

Simular las dependencias async de FastAPI (AsyncMock) no es obvio al inicio.



Mantenimiento continuo

Cada cambio de código puede exigir actualizar los tests correspondientes.



No cubren integración

Las pruebas unitarias no reemplazan pruebas E2E contra la base de datos real.

Lo que nos deja la automatización

01

Integración natural

pytest y Vitest se acoplan al stack sin fricción: uno ya estaba en requirements.txt y el otro reutiliza la configuración de Vite.

02

Inversión razonable

Escribir las 45 pruebas tomó ~3-4 horas; ejecutarlas toma menos de 7 segundos, cuantas veces queramos.

03

Bugs que no vimos

Los tests forzaron a definir casos borde reales, como el stock exactamente igual al mínimo.

04

Confianza para refactorizar

Con la suite en verde, cambiar código deja de ser un acto de fe: cualquier regresión se detecta al instante.